Qiskit v2.x Practice Quiz

Questions

1. What does the following circuit prepare?

```
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
```

   A. $|00\rangle$
   B. $|11\rangle$
   C. $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$
   D. $\frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$

2. Given the code below, how would you retrieve probability distribution from sampler results? Assume all components have been imported correctly.

```
qbits = QuantumRegister(2,"qbits")
clbits = ClassicalRegister(2,"clbits")
qc = QuantumCircuit(qbits, clbits)
(q0, q1) = qbits
(c0, c1) = clbits
qc.h(0)
qc.cx(0,1)
qc.measure(qbits,clbits)
service = QiskitRuntimeService()
backend = service.least_busy()
pm = generate_preset_pass_manager(backend=backend,optimization_level=2)
qc_isa = pm.run(qc)
sampler = SamplerV2(mode=backend)
job =sampler.run([qc_isa])
result = job.result()
```

   A. counts = result[0].data.meas.get_counts()
   B. counts = result[0].data.clbits.get_counts()
   C. counts = result[0].data.evs.get_counts()
   D. counts = result[0].data.stds.get_counts()

3. What does the Sampler primitive primarily return?

   A. Expectation values
   B. Measurement samples or bitstring results

C. Only statevectors

D. Only transpiled circuits

4. Given the following code fragment, what is the approximate probability that a measurement would result in a bit value of 1?

```
from qiskit import QuantumCircuit
from numpy import pi
qc = QuantumCircuit(1)
qc.reset(0)
qc.sx(0)
qc.rx(pi/2,0)
qc.measure_all()
```

A. 0.75

B. 0.0

C. 0.5

D. 1.0

5. When exporting Qiskit code to OpenQASM 3 as a file, which of these method has to be used?

A. export_to_qasm()

B. dumps()

C. qiskit_to_qasm()

D. dump()

6. What does layout refer to in transpilation?

A. Mapping virtual qubits to physical qubits

B. Choosing the number of shots

C. Drawing the circuit

D. Choosing classical register names

7. Which of these options is unique to Sampler??

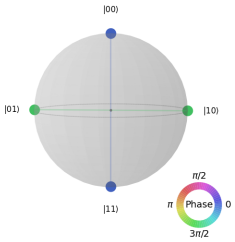
A. default_shots

B. default_precision

C. shots_per_randomization

D. none of the above

8. Which code would generate the following qsphere?



A.
```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_state_qsphere
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0,1)
qc.rz(np.pi/2,1)
qc.draw("mpl")
plot_state_qsphere(qc)
```

B.
```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_state_qsphere
qc = QuantumCircuit(2)
qc.h(0)
qc.rx(np.pi/2,1)
qc.cx(0,1)
qc.draw("mpl")
plot_state_qsphere(qc)
```

C.
```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_state_qsphere
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0,1)
qc.draw("mpl")
plot_state_qsphere(qc)
```

D.
```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_state_qsphere
qc = QuantumCircuit(2)
qc.h(0)
qc.y(1)
qc.rx(np.pi/2,1)
qc.h(0)
plot_state_qsphere(qc)
```

9. Why may SWAP gates be inserted during transpilation?

   A. To reset qubits
   B. To satisfy hardware connectivity constraints
   C. To increase shot count
   D. To remove all noise

10. What does the Estimator primitive primarily compute?

   A. Bitstring counts
   B. Expectation values of observables
   C. Circuit diagrams
   D. Backend coupling maps

11. Which of the following are transpiler stages? *(Select all that apply)*

   A. Layout
   B. Routing
   C. Decomposing
   D. Scheduling

12. OpenQASM is best described as:

    A. A language for representing quantum programs
    B. A physical quantum processor
    C. A Python plotting library
    D. A classical database system

13. Which code correctly creates a two-qubit quantum circuit?

    A. QuantumCircuit()
    B. QuantumCircuit(2)
    C. Circuit(2)
    D. QuantumRegister.create(2)

14. Which of these is not a valid Estimator PUB entry?

    A. a single parametarized quantum circuit
    B. a single quantum circuit that contains no parameters
    C. target precision
    D. number of shots

15. What is the purpose of transpilation?

    A. To convert a circuit into backend-compatible instructions
    B. To measure all qubits automatically
    C. To create a statevector
    D. To define an observable

16. What does the Bloch sphere represent?

    A. Only classical bits
    B. Single-qubit pure states
    C. Only two-qubit entangled states
    D. Classical probability distributions only

17. What does an X gate do to $|0\rangle$?

    A. Creates $|+\rangle$
    B. Maps it to $|1\rangle$
    C. Measures the qubit
    D. Adds a global phase only

18. Which Pauli operator has eigenstates $|+\rangle$ and $|-\rangle$

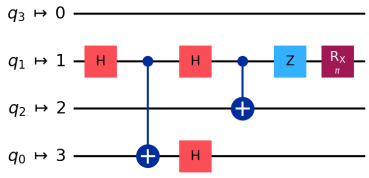
    A. X
    B. Y
    C. Z
    D. I

19. Which of the circuits below is possible output for the code below?

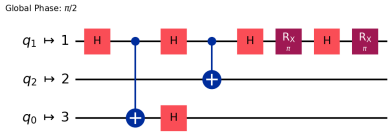
```
from qiskit import *
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit.transpiler.preset_passmanagers import (
  generate_preset_pass_manager, )
from math import pi

qc = QuantumCircuit(4)
qc.h(0)
for i in range(2):
    qc.cx(i,i+1)
qc.z(1)
qc.rx(pi,1)
service = QiskitRuntimeService()
backend = service.least_busy()
pm = generate_preset_pass_manager(
    optimization_level=1,
    basis_gates=["rx","cx","h"],
    coupling_map=[[0,2],[1,3],[1,2]]
)
qc_isa = pm.run(qc)
qc_isa.draw("mpl",idle_wires=True)
```

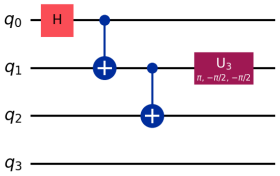
A.



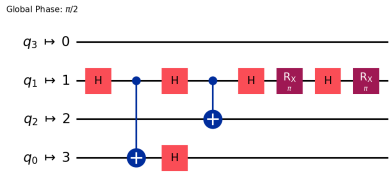
B.



C.



D.



20. Given the code below, how would you retrieve probability distribution from sampler results? Assume all components have been imported correctly.

```
qbits = QuantumRegister(2,"qbits")
clbits = ClassicalRegister(2,"clbits")
qc = QuantumCircuit(qbits, clbits)
(q0, q1) = qbits
(c0, c1) = clbits
qc.h(0)
qc.cx(0,1)
qc.measure_all()
service = QiskitRuntimeService()
backend = service.least_busy()
pm = generate_preset_pass_manager(backend=backend, optimization_level=2)
```

```
qc_isa = pm.run(qc)
sampler = SamplerV2(mode=backend)
job =sampler.run([qc_isa])
result = job.result()
```

A. counts = result[0].data.meas.get_counts()

B. counts = result[0].data.clbits.get_counts()

C. counts = result[0].data.evs.get_counts()

D. counts = result[0].data.stds.get_counts()

Answer Key

1. S3-002: C
2. S7-004: B
3. S5-001: B
4. S1-007: D
5. S8-002: B
6. S4-003: A
7. S5-003: D
8. S2-003: B
9. S4-002: B
10. S6-001: B
11. S3-007: A, B, D
12. S8-001: A
13. S3-001: B
14. S6-005: D
15. S4-001: A
16. S2-001: B
17. S1-001: B
18. S1-004: A
19. S3-006: D
20. S7-005: A